MICROSERVICES WORKFLOW ARCHITECTURE • ENGINEERING GUIDE

Designing Workflows in Microservices: Orchestration vs. Choreography

Decomposition of State Machines, Event Cascades, Sagas, Compensations, and Hybrid Topologies

EXECUTIVE SUMMARY

Distributed workflows span multiple microservices without shared ACID transactions. Organizations face a fundamental architectural choice: centralized command-and-control (Orchestration) vs decentralized reactive events (Choreography). While choreography maximizes service decoupling, orchestration prevents the 'pinball architecture' trap and provides vital operational observability.

1. THE WORKFLOW ARCHITECTURAL DICHOTOMY



2. STRUCTURAL COMPARISON OF PATTERNS

Feature	Orchestration (Temporal / Cadence)	Choreography (Kafka / RabbitMQ)
Control Model	Centralized coordinator holds execution state.	Decentralized; state is distributed across participants.
Communication	Point-to-point commands (gRPC / Queues).	Event publishing / subscribing (Pub-Sub).
Debugging	Simple Full visual execution tree and logs.	Complex Requires end-to-end distributed tracing.
Failure Mode	Single coordinator failure (mitigated by RAFT).	Cyclic loops & unhandled message cascades.

3. THE PINBALL ARCHITECTURE FAILURE IN CHOREOGRAPHY

In naive choreography, business logic bounces between services like a pinball. Changing one event payload produces unexpected regressions across services maintained by unrelated teams.

4. THE SAGA PATTERN & COMPENSATING TRANSACTIONS

When a distributed workflow fails at step k, previously committed steps (1 to k-1) cannot be rolled back with SQL. The system must execute semantic **Compensating Transactions**:

```
// Temporal Go Workflow with Automated Compensations
func OrderFulfillmentWorkflow(ctx workflow.Context, order Order) error {
    var compensations []func(workflow.Context) error

    // Step 1: Charge Credit Card
    err := workflow.ExecuteActivity(ctx, ChargePayment, order).Get(ctx, nil)
    if err != nil { return err }
    compensations = append(compensations, func(ctx workflow.Context) error {
        return workflow.ExecuteActivity(ctx, RefundPayment, order).Get(ctx, nil)
    })

    // Step 2: Reserve Inventory
    err = workflow.ExecuteActivity(ctx, ReserveInventory, order).Get(ctx, nil)
    if err != nil {
        // Rollback all previous steps in reverse order!
        for i := len(compensations) - 1; i >= 0; i-- {
            _ = compensations[i](ctx)
        }
        return fmt.Errorf("inventory reservation failed: %w", err)
    }

    return nil
}
```

5. ORCHESTRATION STATE MACHINES (TEMPORAL / STEP FUNCTIONS)

1. Deterministic Replay

Workflow definitions execute deterministically. On node crash, the engine re-evaluates event history to restore exact in-memory state.

2. Heartbeats & Timeouts

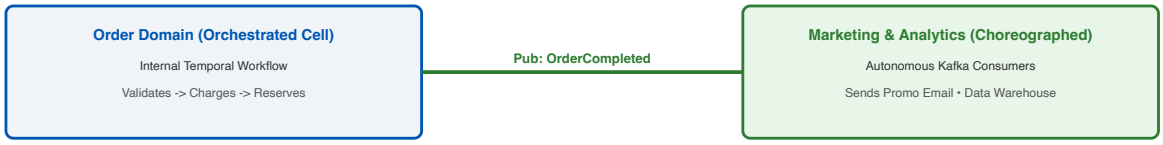
Activities report heartbeats. If a worker dies during transcoding or payment settlement, the orchestrator re-routes the task.

3. Versioning Invariants

Workflows are long-lived (can run for weeks). Engine supports side-by-side execution of v1 and v2 workflow code.

6. THE HYBRID TOPOLOGY: ORCHESTRATED CELLS, CHOREOGRAPHED MESH

Enterprise architectures combine both paradigms to balance local clarity with organizational independence:



7. ARCHITECTURAL DECISION MATRIX: WHICH PATTERN TO CHOOSE?

Choose Orchestration When:

- Workflow has > 3 sequential or conditional steps.
- Regulatory auditing requires visualizing the state machine.
- Workflow spans hours, days, or involves human approval gates.
- Rollback and compensation logic must be centralized.

Choose Choreography When:

- One-way broadcast event notifications (1 producer to many consumers).
- Subscribers have zero impact on the publisher's success.
- Teams demand complete operational decoupling.
- Workflows are strictly 1-2 steps with no compensation requirements.

8. PRODUCTION VERIFICATION CHECKLIST

Workflow Architecture Gates

1. **Idempotent Compensation:** Every rollback endpoint must safely accept duplicate calls without throwing errors.
2. **End-to-End Tracing:** Inject W3C `traceparent` into all workflow commands and broadcast events.
3. **Workflow Timeouts:** Enforce strict timeouts on every activity invocation to prevent zombie executions.
4. **Schema Governance:** Backward-compatibility tests required for all orchestrator task schemas.